FIAP ON

DJANGO WEB FRAMEWORK

ROTAS E VIEWS

ANDREY MASIERO

3

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 3.1 – Resultado do layout construído para a página inicial do sistema.....8
Figura 3.2 – Tela inicial do sistema para clientes.....9
Figura 3.3 – Tela inicial após a inclusão do método index.....11
Figura 3.4 – Template inserido com suscesso.....12
Figura 3.5 – Execução do comando collectstatic.....14
Figura 3.6 – Diretórios criados após a execução do comando collectstatic.....15
Figura 3.7 – Imagens carregadas com sucesso.....16
Figura 3.8 – Tela de detalhes do livro.....20
Figura 3.9 – Livros buscados a partir do banco de dados.....27
Figura3.10 – Detalhes do livro apresentado ao cliente.....30

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 3.1 – Código HTML que servirá como base para as demais páginas 8
Código-fonte 3.2 – Código de configuração do endereço de acesso ao sistema 10
Código-fonte 3.3 – Inclusão do método index no arquivo views.py 10
Código-fonte 3.4 – Inclusão do método index no arquivo views.py 11
Código-fonte 3.5 – Alteração do views.py para uso da função render 12
Código-fonte 3.6 – Configuração de templates no sistema 13
Código-fonte 3.7 – Configuração de templates no Sistema 14
Código-fonte 3.8 – Página de detalhes dos livros 19
Código-fonte 3.9 – Inclusão da nova rota para detalhes do livro no arquivo urls.py 19
Código-fonte 3.10 – Inclusão do método livro para renderizar os detalhes sobre um livro 19
Código-fonte 3.11 – Inclusão do método livro para renderizar os detalhes sobre um livro 20
Código-fonte 3.12 – Código do template de apoio base.html 22
Código-fonte 3.13 – Inclusão do método livro para renderizar os detalhes sobre um livro 24
Código-fonte 3.14 – Conteúdo do arquivo cabecalho.html 25
Código-fonte 3.15 – Recuperando os livros do banco de dados 26
Código-fonte 3.16 – Criando os livros com base nos dados 27
Código-fonte 3.17 – Criando os livros com base nos dados 28
Código-fonte 3.18 – Recuperando um livro específico do banco de dados 28
Código-fonte 3.19 – Detalhes dinâmicos apresentados no template 29

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

3 ROTAS E VIEWS 5
3.1 Apresentando a página ao cliente 5
3.2 Construindo a rota raiz 9
3.3 Trabalhando com template 11
3.4 Arquivos estáticos 13
3.5 Um livro interessante 16
3.6 Reaproveitando o HTML 21
3.7 Vamos por partes 24
3.8 Mostre o que está na estante 26
3.9 Vendo o livro em detalhes 27
REFERÊNCIAS 31

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 ROTAS E VIEWS

3.1 Apresentando a página ao cliente

Com a finalização da modelagem dos objetos de negócios e com a parte administrativa do sistema configuradas, precisamos começar a trabalhar na camada de apresentação para o cliente. Essa é a parte em que os clientes da biblioteca terão acesso ao acervo de livros disponíveis ao longo do uso. Para iniciar esse processo, precisamos estabelecer uma base de layout criada com HTML, Bootstrap, para facilitar a parte de estilização e FontAwesome para a parte de iconografia do site.

Vamos começar criando um arquivo HTML, fora do projeto, apenas para definirmos a parte visual. Quando a parte visual estiver completa, a incluiremos no projeto Django e iniciaremos a integração com as demais partes do sistema. O código-fonte “Código HTML que servirá como base para as demais páginas” apresenta o código da página index.

```
<!DOCTYPE html>
<html lang="pt-br">
     <head>
          <meta charset="UTF-8" />
          <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
          <title>Biblioteca</title>
          <link
               rel="stylesheet"
     href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/c
ss/bootstrap.min.css"
          />
          <link
               rel="stylesheet"
     href="https://use.fontawesome.com/releases/v5.7.0/css/all.
css"
          />
     </head>
     <body>
          <div class="navbar navbar-expand-lg navbar-light bg-
light">
               <header class="container">
                    <a class="navbar-brand" href="#">
                         <i class="fas fa-book-open"></i>
                         Biblioteca
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                    </a>
                    <button
                         class="navbar-toggler"
                         type="button"
                         data-toggle="collapse"
                         data-target="#navbarNavAltMarkup"
                         aria-controls="navbarNavAltMarkup"
                         aria-expanded="false"
                         aria-label="Toggle navigation"
                    >
                         <span class="navbar-toggler-
icon"></span>
                    </button>
                    <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
                         <div class="navbar-nav">
                              <a class="nav-link"
href="#">Livros</a>
                              <a class="nav-link"
href="#">Reserva</a>
                         </div>
                    </div>
               </header>
          </div>
          <main class="container">
               <form class="form-inline">
                    <div class="input-group mt-5 mb-5 col-12">
                         <input
                              type="text"
                              class="form-control"
                              placeholder="Busque pelo nome do
livro"
                         />
                         <div class="input-group-append">
                              <div class="input-group-text">
                                   <a class="text-dark"
href="#">
                                        <i class="fas fa-
search"></i>
                                   </a>
                              </div>
                         </div>
                    </div>
               </form>
               <div class="col-12 d-flex flex-row flex-wrap
justify-content-center">
                    <div class="card mr-4 mb-3" style="width:
18rem">
                         <img src="book.png" class="card-img-
top" />
                         <div class="card-body">
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                <h5 class="card-
title">Titulo</h5>
                                <p class="card-text">
                                     Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
                                     Suspendisse aliquet eros
tempor odio tristique congue.
                                </p>
                                <a href="#" class="card-
link">Detalhes</a>
                           </div>
                      </div>
                      <div class="card mr-4 mb-3" style="width:
18rem">
                           <img src="book.png" class="card-img-
top" />
                           <div class="card-body">
                                <h5 class="card-
title">Titulo</h5>
                                <p class="card-text">
                                     Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
                                     Suspendisse aliquet eros
tempor odio tristique congue.
                                </p>
                                <a href="#" class="card-
link">Detalhes</a>
                           </div>
                      </div>
                      <div class="card mr-4 mb-3" style="width:
18rem">
                           <img src="book.png" class="card-img-
top" />
                           <div class="card-body">
                                <h5 class="card-
title">Titulo</h5>
                                <p class="card-text">
                                     Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
                                     Suspendisse aliquet eros
tempor odio tristique congue.
                                </p>
                                <a href="#" class="card-
link">Detalhes</a>
                           </div>
                      </div>
                 </div>
            </main>
            <script
                 src="https://code.jquery.com/jquery-
3.5.1.slim.min.js"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            ></script>
            <script

      src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/um
d/popper.min.js"
            ></script>
            <script

      src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js
/bootstrap.min.js"
            ></script>
      </body>
</html>
```

Código-fonte 3.1 – Código HTML que servirá como base para as demais páginas
Fonte: Elaborado pelo autor

Após a criação do arquivo base, podemos executar no navegador para conferir o resultado obtido com a construção da página index e o posicionamento de seus elementos. Confira o resultado esperado através da figura “Resultado do layout construído para a página inicial do sistema”.

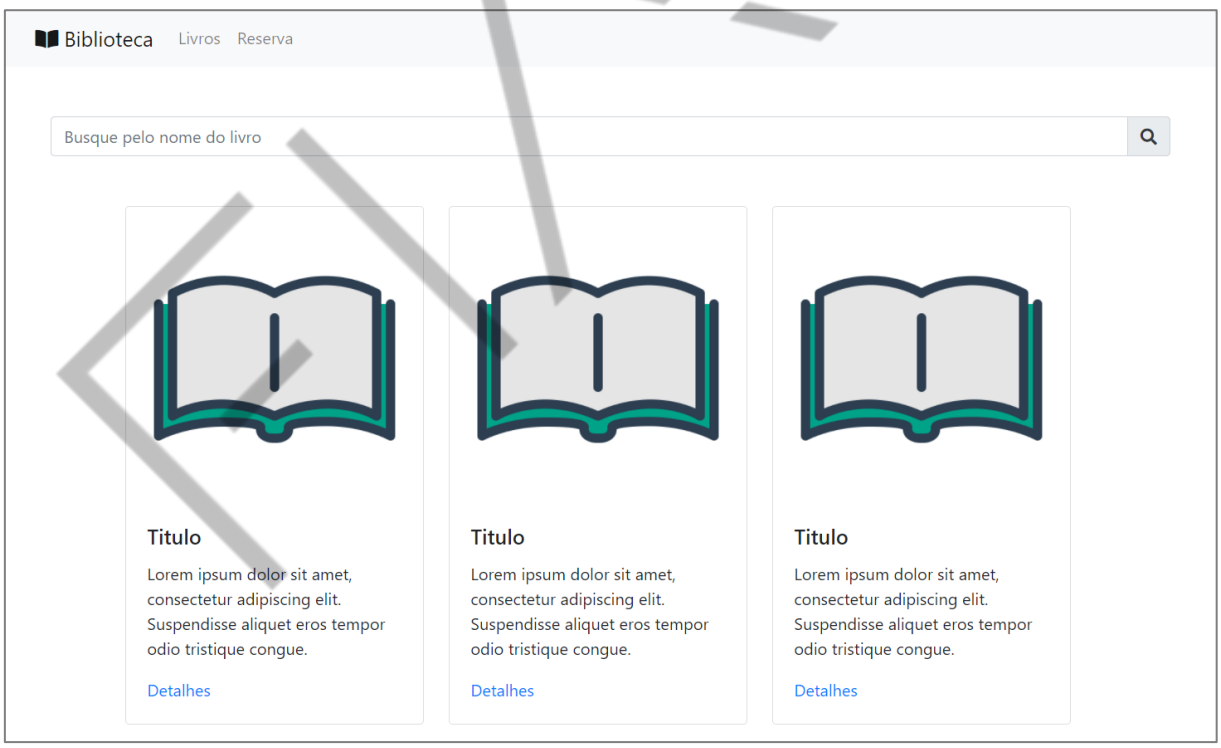

Figura 3.1 – Resultado do layout construído para a página inicial do sistema
Fonte: Elaborado pelo autor

É importante que fique claro que o estilo aplicado no site não é o foco do material. Portanto, a página apresentada na figura “Resultado do layout construído para a página inicial do sistema” pode apresentar alguns pontos de melhoria para atender algumas premissas de design.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A partir desse ponto, podemos começar a configurar o projeto desenvolvido em Django para atender ao sistema de uso por parte do cliente final.

3.2 Construindo a rota raiz

Ao iniciar o script de inicialização do servidor, e na sequência acessar o endereço http://localhost:8000/ é esperado acessar a página inicial do sistema. Contudo, se executarmos nesse momento, temos a imagem apresentada na figura "Tela inicial do sistema para clientes".

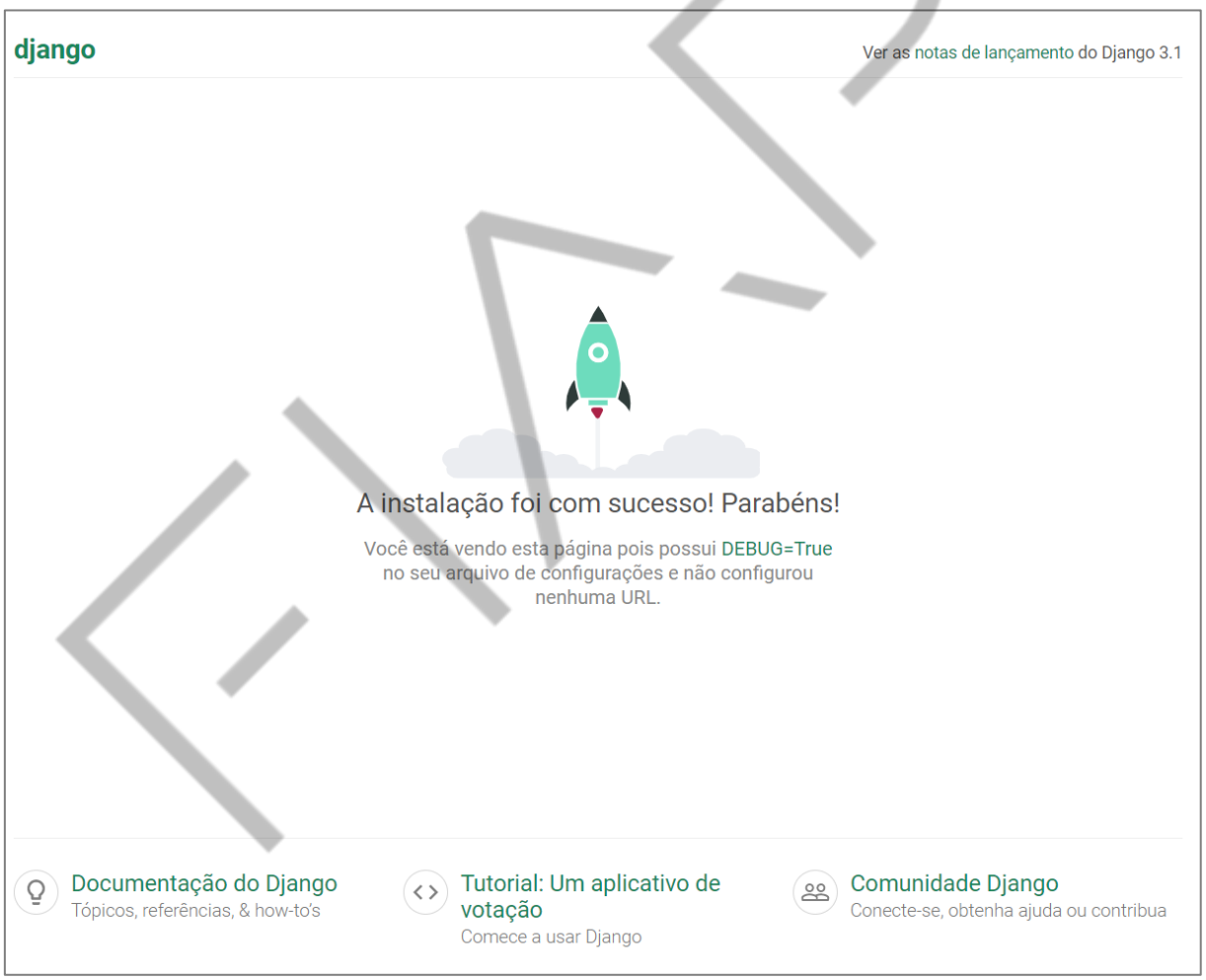

Figura 3.2 – Tela inicial do sistema para clientes
Fonte: Elaborado pelo autor

Isso acontece, pois não configuramos o recurso retornado através do acesso à rota (endereço) raiz do sistema. Para começarmos a personalizar esse resultado, vamos criar um arquivo chamado urls.py, que terá a configuração de todos os caminhos do nosso aplicativo de livros, a princípio. Ele deverá ser colocado na raiz

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

do app de livros. Vamos inserir o código-fonte “Código de configuração do endereço de acesso ao sistema” no arquivo criado.

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index')
]
```

Código-fonte 3.2 – Código de configuração do endereço de acesso ao sistema
Fonte: Elaborador pelo autor

Esse arquivo é responsável por todas as urls que desejamos acessar em nosso sistema. Para criar o endereço de acesso, vamos utilizar a função de path, que é importada através do pacote django.urls. Importaremos também o arquivo views.py, que é responsável pela manipulação ou comportamento do sistema, na criação dos recursos visuais que serão exibidos na tela.

Criaremos uma lista chamada urlpatterns que contém todos os endereços criados através da função path. O primeiro parâmetro é nome da rota que queremos acessar; o segundo é o método que responderá àquela rota; e o terceiro é o nome dentro do aplicativo para esse conjunto de rotas. Deixamos vazio o primeiro parâmetro, pois queremos acessá-lo a partir do endereço base do site.

O próximo passo é a criação do método index dentro do arquivo views.py. Ele retornará o recurso que desejamos ao acessar a rota configurada. O código-fonte “Inclusão do método index no arquivo views.py” apresenta o arquivo views.py com a inclusão do método index.

```
from django.shortcuts import render
from django.http import HttpResponse

def index(request):
    return HttpResponse('<h1>Biblioteca Digital</h1>')
```

Código-fonte 3.3 – Inclusão do método index no arquivo views.py
Fonte: Elaborado pelo autor

O método que tratará a rota, nesse caso index, recebe como parâmetro um objeto de request. Esse objeto carrega todas as informações enviadas pelo cliente no momento de acesso à rota. Para devolvermos a resposta ao cliente, utilizaremos um objeto HttpResponse. Nesse primeiro momento, vamos retornar uma String com

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

o código HTML desejado apenas para realizarmos um teste. E por fim, no app principal (library), vamos até o arquivo urls.py para incluir o arquivo urls.py de livros. Veja o código-fonte “Inclusão do método index no arquivo views.py” para conferir o resultado da alteração.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('', include('livros.urls')),
    path('admin/', admin.site.urls),
]
```

Código-fonte 3.4 – Inclusão do método index no arquivo views.py
Fonte: Elaborado pelo autor

Agora ao acessarmos o endereço principal do site da nossa biblioteca, http://localhost:8000/, obteremos o resultado apresentado na figura “Tela inicial após a inclusão do método index”.

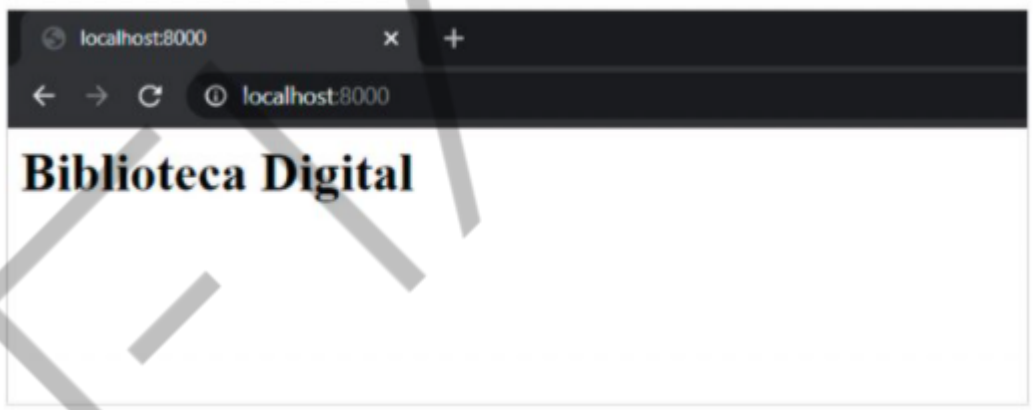

Figura 3.3 – Tela inicial após a inclusão do método index
Fonte: Elaborado pelo autor

Pronto, conseguimos acessar as informações criadas a partir do método index, entretanto, não é legal escrevermos todos os elementos HTML da página através dessa String. Vamos solucionar isso na próxima seção.

3.3 Trabalhando com template

Para não termos que devolver todo o código HTML dentro do objeto HttpResponse, vamos utilizar o arquivo index.html criado no início do capítulo para

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

apresentar ao usuário. Utilizaremos a função render, que está disponível no pacote django.shortcuts. Alterando o código do views.py de livros, teremos o código-fonte “Alteração do views.py para uso da função render”, apresentado a seguir.

```
from django.shortcuts import render


def index(request):
    return render(request, 'index.html')
```

Código-fonte 3.5 – Alteração do views.py para uso da função render
Fonte: Elaborado pelo autor

O método render recebe como primeiro parâmetro o objeto de request, recebido por index inicialmente e o segundo o nome do template HTML. Para que o Django encontre o template, devemos inserir o arquivo HTML em uma pasta chamada templates. Assim, o render inicia a procura de um arquivo com o nome passado, sempre a partir dessa pasta.

O resultado obtido é apresentado através da figura “Template inserido com sucesso”.

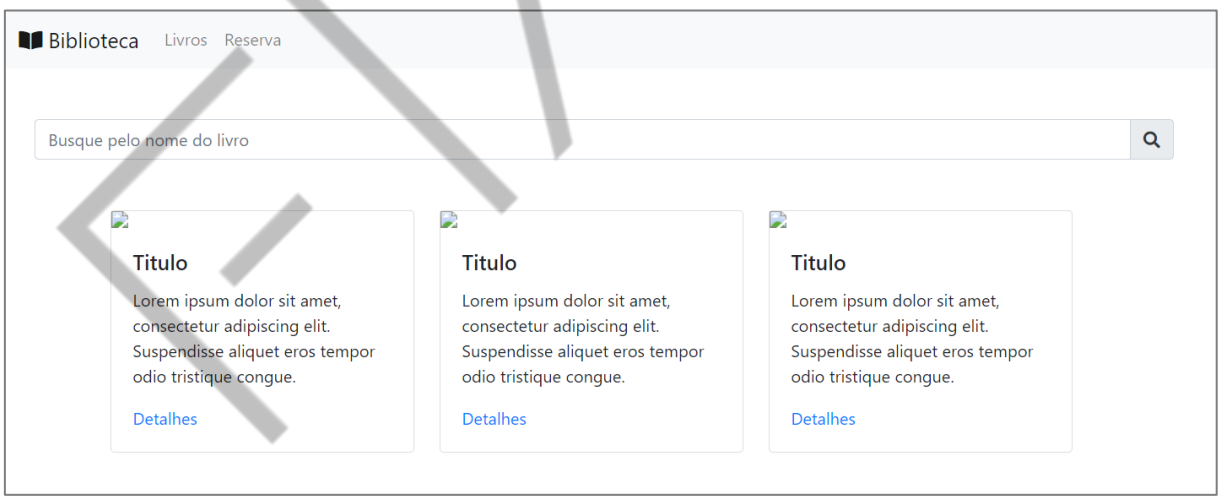

Figura 3.4 – Template inserido com sucesso
Fonte: Elaborado pelo autor

Apesar de nosso template ter sido inserido com sucesso, as imagens ainda não foram incluídas. Aparecem os ícones de não encontradas para os três livros que são apresentados na tela. A solução para as imagens é a adição do arquivo desejado, contudo, arquivos de imagens, assim como os de estilos, scripts e outros

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

são considerados arquivos estáticos ao servidor. Trabalharemos com eles na próxima seção e solucionar esse problema.

3.4 Arquivos estáticos

Como sabemos, arquivos de scripts, estilo e imagens são considerados arquivos estáticos. Para o servidor encontrá-los é necessário saber como é o caminho absoluto de cada um deles. Assim, a referência aos arquivos chega de maneira adequada ao cliente.

O Django já possui alguns pacotes que auxiliam a tratativa de arquivos estáticos nos projetos. O pacote que faz essa gestão dos estáticos é o django.contrib.staticfiles. Vamos começar a configuração no arquivo settings.py, do aplicativo principal. Procure pela lista TEMPLATES, na qual colocaremos o caminho de onde estão os templates do nosso aplicativo. Vamos alterar o objeto DIRS dentro de TEMPLATES. Deixe de acordo com o código-fonte “Configuração de templates no sistema”.

```
TEMPLATES = [
    {
        'BACKEND':
'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'livros/templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',

'django.contrib.auth.context_processors.auth',

'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Código-fonte 3.6 – Configuração de templates no sistema
Fonte: Elaborado pelo autor

Utilizamos o módulo os para unir o endereço base da aplicação com o endereço específico até chegar aos templates. O próximo passo é informar ao Django onde ficam os arquivos estáticos. Ao final do arquivo settings.py, vamos

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

adicionar STATIC_ROOT, que será o local de coleta para os clientes dos arquivos estáticos. Precisamos definir também o STATIC_URL, que deverá ser como estará referenciado nos templates, e o STATICFILES_DIRS, que é uma lista dos diretórios que armazenam os arquivos estáticos, geralmente dentro de uma das aplicações do projeto. O resultado dessa implementação deve ser o mesmo apresentado no código-fonte “Configuração de templates no sistema”.

```
STATIC_ROOT = os.path.join(BASE_DIR, 'static')
STATIC_URL = '/static/'
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'library/static')
]
```

Código-fonte 3.7 – Configuração de templates no Sistema
Fonte: Elaborado pelo autor

Vamos criar uma pasta static dentro de library, e nela colocaremos a imagem do livro em nosso layout. Agora, precisamos informar ao Django que temos arquivos estáticos. E vamos utilizar o comando collectstatic para nos ajudar no referenciamento da imagem. Execute o comando `python manager.py collectstatic`, para que o processo ocorra com sucesso. O comando deve ser executado via terminal, com o devido ambiente virtual carregado. O resultado é apresentado na figura “Execução do comando collectstatic”.

```
D:\django\library-app\library (main -> origin)
(venv)   python manage.py collectstatic

133 static files copied to 'D:\django\library-app\library\static'.
```

Figura 3.5 – Execução do comando collectstatic
Fonte: Elaborado pelo autor

Na estrutura do projeto, um novo diretório, irmão de library, é criado e é chamado de static, e dentro dele teremos todos os arquivos estáticos tanto da aplicação cliente que estamos criando quanto do admin, conforme apresentado na figura “Diretórios criados após a execução do comando collectstatic”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

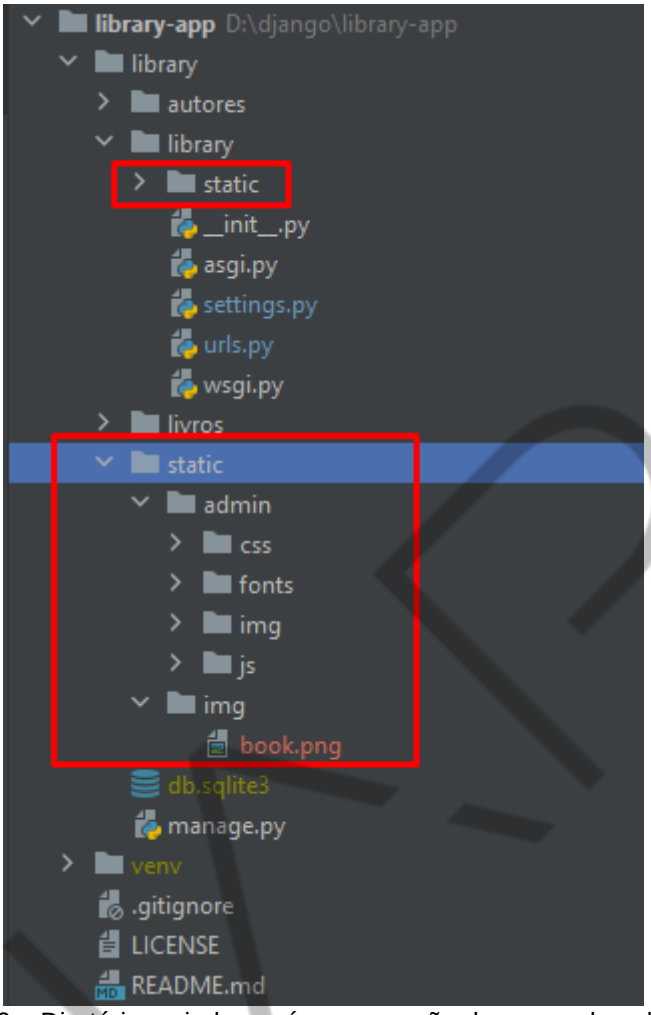

Figura 3.6 – Diretórios criados após a execução do comando collectstatic
Fonte: Elaborado pelo autor

Agora que os arquivos estáticos estão disponíveis, devemos carregá-los para torná-los acessíveis no HTML. Utilizaremos um comando `{% load static %}` no início do template. Isso fará com que estejam disponíveis para acesso no HTML.

Com a expressão regular, vamos referenciar as imagens nos atributos src da tag img com o seguinte código `{% static 'img/book.png' %}`. O resultado deve ser o apresentado na figura “Imagens carregadas com sucesso”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

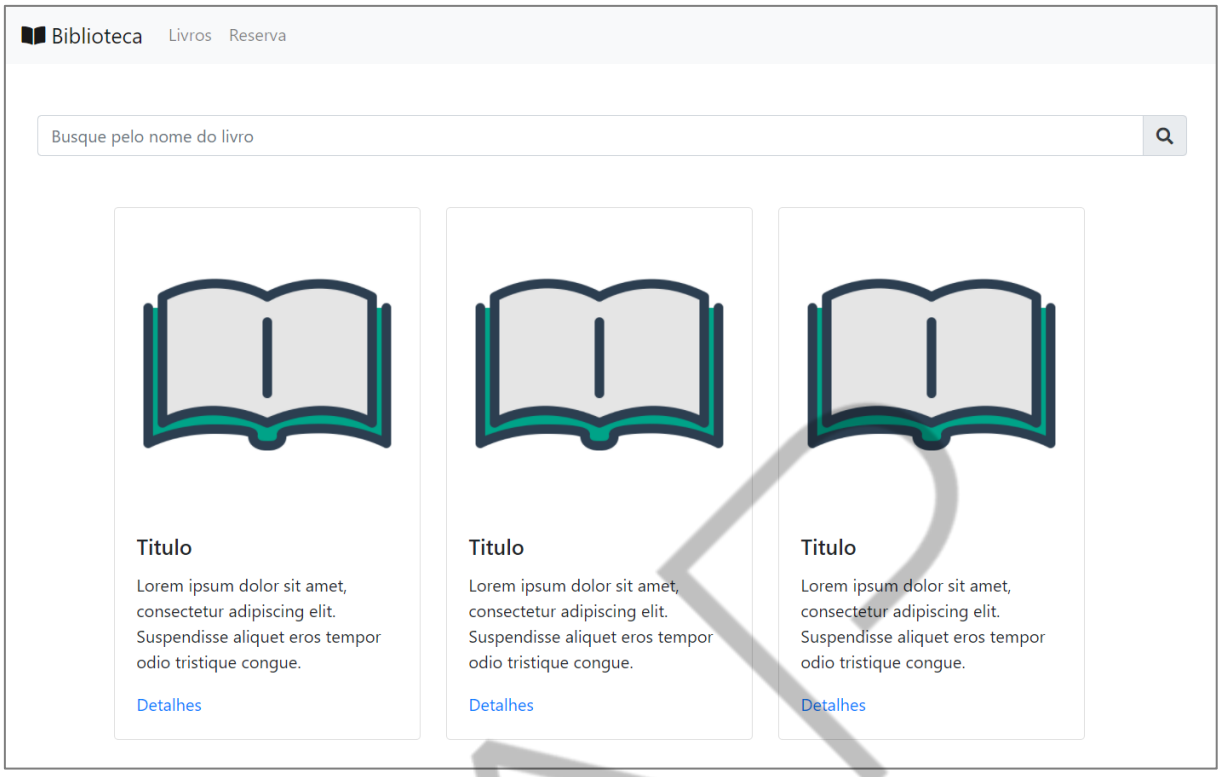

Figura 3.7 – Imagens carregadas com sucesso
Fonte: Elaborado pelo autor

Vamos agora criar as navegações do menu, com um template para detalhes sobre o livro.

3.5 Um livro interessante

O primeiro passo é criarmos o arquivo livro.html. O código-fonte “Página de detalhes dos livros” apresenta o código do HTML para os detalhes dos livros.

```
{% load static %}
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width,
initial-scale=1.0"/>
    <title>Biblioteca</title>
    <link
            rel="stylesheet"

href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/b
ootstrap.min.css"
    />
    <link
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            rel="stylesheet"

href="https://use.fontawesome.com/releases/v5.7.0/css/all.css"
    />
</head>
<body>
<div class="navbar navbar-expand-lg navbar-light bg-light">
    <header class="container">
        <a class="navbar-brand" href="#">
            <i class="fas fa-book-open"></i>
            Biblioteca
        </a>
        <button
                class="navbar-toggler"
                type="button"
                data-toggle="collapse"
                data-target="#navbarNavAltMarkup"
                aria-controls="navbarNavAltMarkup"
                aria-expanded="false"
                aria-label="Toggle navigation"
        >
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
            <div class="navbar-nav">
                <a class="nav-link" href="#">Livros</a>
                <a class="nav-link" href="#">Reserva</a>
            </div>
        </div>
    </header>
</div>
<main class="container">
    <form class="form-inline">
        <div class="input-group mt-5 mb-5 col-12">
            <input
                    type="text"
                    class="form-control"
                    placeholder="Busque pelo nome do livro"
            />
            <div class="input-group-append">
                <div class="input-group-text">
                    <a class="text-dark" href="#">
                        <i class="fas fa-search"></i>
                    </a>
                </div>
            </div>
        </div>
    </form>
    <section class="col-12">
        <div class="d-flex flex-row">
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            <img src="{% static 'img/book.png'%}" class="img-
fluid img-thumbnail rounded" style="height: 250px"/>
            <div class="flex-column ml-4">
                <h1>Titulo</h1>
                <h2>Autor</h2>
            </div>
        </div>
        <div class="d-flex flex-row">
            <p class="col-8">Lorem ipsum dolor sit amet,
consectetur adipiscing elit.Suspendisse aliquet eros tempor
                odio tristique congue.Lorem ipsum dolor sit
amet, consectetur adipiscing elit. Nullam in magna sed
                libero viverra scelerisque. Nam efficitur
justo dolor. Quisque vel dui fermentum, posuere orci non,
                vehicula elit. Cras nulla ipsum, vulputate
faucibus venenatis sit amet, malesuada at sem. Pellentesque
                at felis sit amet dolor sollicitudin rhoncus
eu vitae massa. Donec tincidunt magna sed hendrerit
                ultrices. Integer a enim quis ex rhoncus
faucibus et ac dui. Nullam vel ipsum erat. Morbi tristique est
                eget dui pharetra pulvinar. Etiam eu magna et
libero feugiat tincidunt eget at sapien. Duis sagittis
                efficitur massa ut tincidunt.</p>
            <ul class="list-group ml-3 col-4">
                <li class="list-group-item">Páginas: 133</li>
                <li class="list-group-item">Editora: Editora
X</li>
                <li class="list-group-item">Edição: 9</li>
                <li class="list-group-item">Ano da publicação:
2020</li>
                <li class="list-group-item">Idioma:
Português</li>
                <li class="list-group-item">ISBN:
9788532311443</li>
            </ul>
        </div>

    </section>
</main>
<script
        src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
></script>
<script

src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/po
pper.min.js"
></script>
<script

src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/boo
tstrap.min.js"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
></script>
</body>
</html>
```

Código-fonte 3.8 – Página de detalhes dos livros
Fonte: Elaborado pelo autor

Após a criação do layout das informações detalhadas do livro, precisamos ir em urls.py para adicionar o endereço de chamada do livro. Confira o código-fonte “Inclusão da nova rota para detalhes do livro no arquivo urls.py”.

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('livro', views.livro, name='livro'),
]
```

Código-fonte 3.9 – Inclusão da nova rota para detalhes do livro no arquivo urls.py
Fonte: Elaborado pelo autor

O próximo passo é escrever o método para renderizar o template de detalhes do livro, no arquivo views.py, conforme apresentado através do código-fonte “Inclusão do método livro para renderizar os detalhes sobre um livro”.

```
from django.shortcuts import render


def index(request):
    return render(request, 'index.html')


def livro(request):
    return render(request, 'livro.html')
```

Código-fonte 3.10 – Inclusão do método livro para renderizar os detalhes sobre um livro
Fonte: Elaborado pelo autor

Após todas as configurações e criações realizadas e garantirmos o acesso ao detalhe do livro, podemos conferir o resultado apresentado ao cliente, de acordo com a figura “Tela de detalhes do livro”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

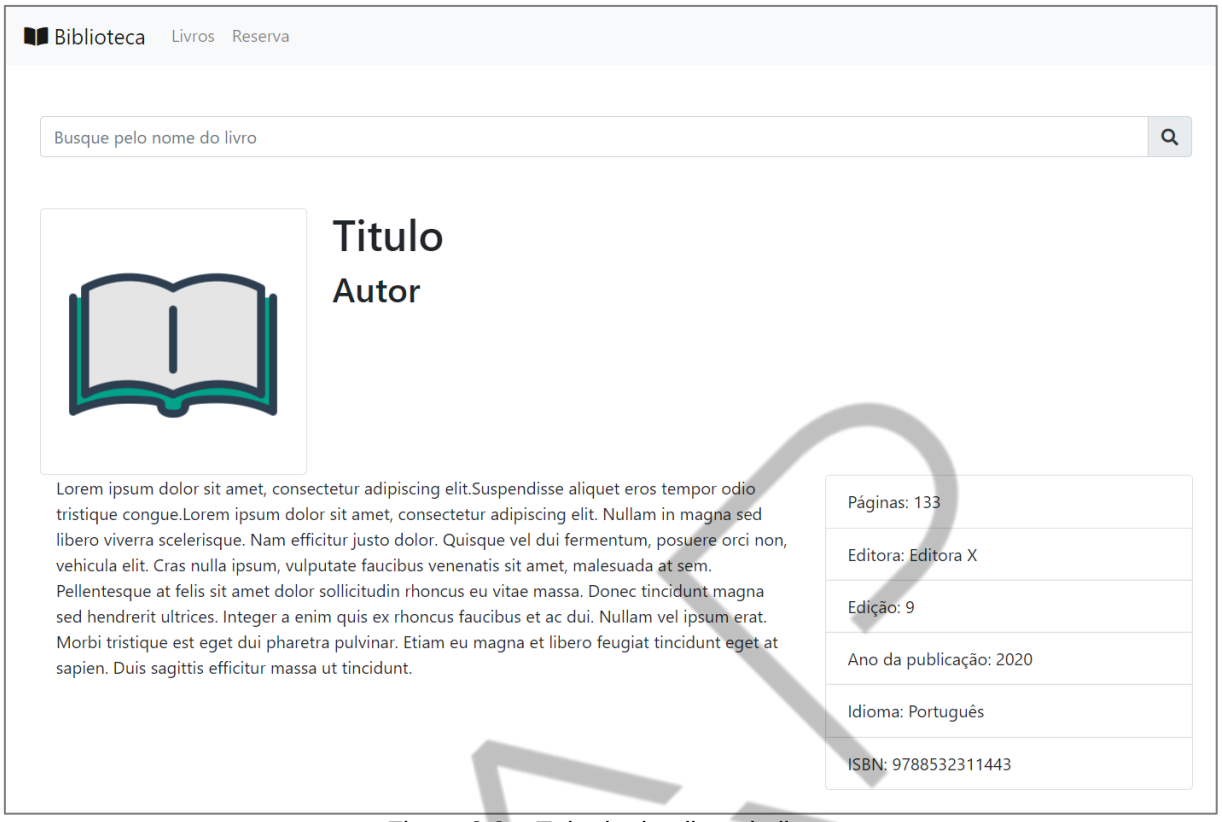

Figura 3.8 – Tela de detalhes do livro
Fonte: Elaborado pelo autor

A partir desse momento, podemos alterar o menu para que os links correspondam à rota determinada para cada um. Confira a alteração através do código-fonte “Inclusão do método livro para renderizar os detalhes sobre um livro”.

```
<div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
    <div class="navbar-nav">
        <a class="nav-link" href="{% url 'index'
%}">Livros</a>
        <a class="nav-link" href="{% url 'livro'
%}">Reserva</a>
    </div>
</div>
```

Código-fonte 3.11 – Inclusão do método livro para renderizar os detalhes sobre um livro
Fonte: Elaborado pelo autor

Pronto, temos a navegação garantida. Porém nosso código está se repetindo demais. Precisamos melhorar isso com uma otimização do template.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.6 Reaproveitando o HTML

No arquivo index.html, fazemos o carregamento dos arquivos estáticos e suas importações. Observando o arquivo livro.html temos um código. Para evitar isso e deixar nosso código mais elegante, criamos um arquivo básico que recebe toda a parte inicial de ambos os códigos até as importações ao final de <body>. A partir dele, poderemos reaproveitar os recursos em outras páginas.

Dentro da pasta templates, criamos o arquivo e nomeamos de base.html. Em index.html, recortamos todo o intervalo de código desde load static até <body>, ficando somente a partir de Preloader, e colamos no novo arquivo. Em seguida, recortamos todo o trecho final de JavaScript e colamos em base.html.

Para indicar que queremos inserir pedaços de código de outras páginas html em posições do arquivo base.html, usamos código Python para block contente. Com esse recurso, indicamos que existe um bloco de conteúdo passado para o trecho em questão encerrado por endblock após <body> e antes dos arquivos JavaScript. O código-fonte “Código do template de apoio base.html” apresenta o resultado do arquivo base.html.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width,
initial-scale=1.0"/>
    <title>Biblioteca</title>
    <link
            rel="stylesheet"

href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/b
ootstrap.min.css"
    />
    <link
            rel="stylesheet"

href="https://use.fontawesome.com/releases/v5.7.0/css/all.css"
    />
</head>
<body>

{% block conteudo %} {% endblock %}
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<script
        src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
></script>
<script

src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/po
pper.min.js"
></script>
<script

src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/boo
tstrap.min.js"
></script>
</body>
</html>
```

Código-fonte 3.12 – Código do template de apoio base.html
Fonte: Elaborado pelo autor

Agora, vamos alterar o arquivo index.html para utilizar esse template base na criação de seu layout. O novo index.html é apresentado através do código-fonte "Inclusão do método livro para renderizar os detalhes sobre um livro".

```
{% extends 'base.html' %}
{% load static %}
{% block conteudo %}
<div class="navbar navbar-expand-lg navbar-light bg-light">
    <header class="container">
        <a class="navbar-brand" href="{% url 'index' %}">
            <i class="fas fa-book-open"></i>
            Biblioteca
        </a>
        <button
                class="navbar-toggler"
                type="button"
                data-toggle="collapse"
                data-target="#navbarNavAltMarkup"
                aria-controls="navbarNavAltMarkup"
                aria-expanded="false"
                aria-label="Toggle navigation"
        >
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
            <div class="navbar-nav">
                <a class="nav-link" href="{% url 'index'
%}">Livros</a>
                <a class="nav-link" href="{% url 'livro'
%}">Reserva</a>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            </div>
        </div>
    </header>
</div>
<main class="container">
    <form class="form-inline">
        <div class="input-group mt-5 mb-5 col-12">
            <input
                    type="text"
                    class="form-control"
                    placeholder="Busque pelo nome do livro"
            />
            <div class="input-group-append">
                <div class="input-group-text">
                    <a class="text-dark" href="#">
                        <i class="fas fa-search"></i>
                    </a>
                </div>
            </div>
        </div>
    </form>
    <div class="col-12 d-flex flex-row flex-wrap justify-
content-center">
        <div class="card mr-4 mb-3" style="width: 18rem">
            <img src="{% static 'img/book.png'%}"
class="card-img-top"/>
            <div class="card-body">
                <h5 class="card-title">Titulo</h5>
                <p class="card-text">
                    Lorem ipsum dolor sit amet, consectetur
adipiscing elit.
                    Suspendisse aliquet eros tempor odio
tristique congue.
                </p>
                <a href="#" class="card-link">Detalhes</a>
            </div>
        </div>
        <div class="card mr-4 mb-3" style="width: 18rem">
            <img src="{% static 'img/book.png'%}"
class="card-img-top"/>
            <div class="card-body">
                <h5 class="card-title">Titulo</h5>
                <p class="card-text">
                    Lorem ipsum dolor sit amet, consectetur
adipiscing elit.
                    Suspendisse aliquet eros tempor odio
tristique congue.
                </p>
                <a href="#" class="card-link">Detalhes</a>
            </div>
        </div>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        <div class="card mr-4 mb-3" style="width: 18rem">
            <img src="{% static 'img/book.png'%}"
class="card-img-top"/>
            <div class="card-body">
                <h5 class="card-title">Titulo</h5>
                <p class="card-text">
                    Lorem ipsum dolor sit amet, consectetur
adipiscing elit.
                    Suspendisse aliquet eros tempor odio
tristique congue.
                </p>
                <a href="#" class="card-link">Detalhes</a>
            </div>
        </div>
    </div>
</main>
{% endblock %}
```

Código-fonte 3.13 – Inclusão do método livro para renderizar os detalhes sobre um livro
Fonte: Elaborado pelo autor

O primeiro passo é informar a partir de qual arquivo o index.html será estendido, ou baseado. Depois, carregamos os blocos estáticos, para as imagens serem carregadas. Não basta o load static ser invocado no arquivo de base. E por fim, com o escopo do block determinado, inserimos o código que irá completar o conteúdo desejado.

Com tudo isso funcionando, ainda existe um ponto que podemos melhorar. O cabeçalho está sendo construído duas vezes, mas não colocaremos no arquivo base, apesar de ser uma estratégia válida. Utilizaremos uma estratégia de reaproveitamento de pequenos trechos de código.

3.7 Vamos por partes

Agora que já sabemos como criar extensões de arquivos para servir de base para outros arquivos HTML, podemos expandir esse conceito para termos pequenos trechos de código que vão compor uma página inteira. Esse é um conceito que conhecemos no Django como partials.

Um bom lugar para aplicarmos esse conceito em nosso código é o menu de navegação. Perceba que ele é um código repetitivo, porém com uma estrutura volátil. Ele pode ser alterado para inclusão e remoção de itens e para alteração das

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

rotas dos links. Essas alterações devem refletir em todas as páginas que possuem o menu. Vamos criar uma partial para solucionar esse problema.

Dentro da pasta de templates, crie uma pasta chamada partials. Crie um arquivo dentro da nova pasta chamado cabecalho.html. Nele, insira toda a tag header. O conteúdo deve ficar de acordo com o código-fonte “Conteúdo do arquivo cabecalho.html”.

```
<header class="container">
    <a class="navbar-brand" href="{% url 'index' %}">
        <i class="fas fa-book-open"></i>
        Biblioteca
    </a>
    <button
            class="navbar-toggler"
            type="button"
            data-toggle="collapse"
            data-target="#navbarNavAltMarkup"
            aria-controls="navbarNavAltMarkup"
            aria-expanded="false"
            aria-label="Toggle navigation"
    >
        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse"
id="navbarNavAltMarkup">
        <div class="navbar-nav">
            <a class="nav-link" href="{% url 'index'
%}">Livros</a>
            <a class="nav-link" href="{% url 'livro'
%}">Reserva</a>
        </div>
    </div>
</header>
```

Código-fonte 3.14 – Conteúdo do arquivo cabecalho.html
Fonte: Elaborado pelo autor

No local da página index.html, onde existia o cabeçalho, devemos inserir a seguinte instrução `{% include 'partials/cabecalho.html' %}`. Agora temos o sistema organizado, sem a necessidade de repetirmos códigos em diferentes arquivos. Com os layouts prontos, precisamos preencher com as informações do banco de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.8 Mostre o que está na estante

Precisamos apresentar as informações de maneira dinâmica, de acordo com as informações que recebemos do banco de dados. Para isso, o primeiro passo é obter todos os objetos livros registrados de acordo com nossa classe modelo. O Django já possui alguns métodos que farão essa consulta para nós, como o caso do objects.all(). Com esses dados, vamos construir um dicionário que será enviado com a função render para que seja possível recuperar as informações em nosso template. O código-fonte “Recuperando os livros do banco de dados” apresenta o arquivo views.py com a alteração na função index.

```
from django.shortcuts import render
from .models import Livro

def index(request):
    dados = {
        'livros': Livro.objects.all()
    }
    return render(request, 'index.html', dados)


def livro(request):
    return render(request, 'livro.html')
```

Código-fonte 3.15 – Recuperando os livros do banco de dados
Fonte: Elaborado pelo autor

Importamos o modelo Livro e a partir dele chamamos a consulta de todos os objetos do banco de dados. Criamos o dicionário de dados e na sequência o enviamos como terceiro parâmetro da função render. Agora utilizaremos um operador do Django, que são duas chaves ({{ }}) para, ao invés de executar o comando como {% %}, exibir a informação na tela. O código da index deve ficar de acordo com o código-fonte “Criando os livros com base nos dados”.

```
<div class="col-12 d-flex flex-row flex-wrap justify-content-
center">
    {% for livro in livros %}
    <div class="card mr-4 mb-3" style="width: 18rem">
        <img src="{% static 'img/book.png'%}" class="card-
img-top"/>
        <div class="card-body">
            <h5 class="card-title">{{ livro.titulo }}</h5>
            <p class="card-text">
                {{ livro.descricao|slice:':150'|add:'...' }}
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                </p>
                <a href="#" class="card-link">Detalhes</a>
            </div>
        </div>
        {% endfor %}
</div>
```

Código-fonte 3.16 – Criando os livros com base nos dados
Fonte: Elaborado pelo autor

O resultado obtido é apresentado na figura “Livros buscados a partir do banco de dados”.

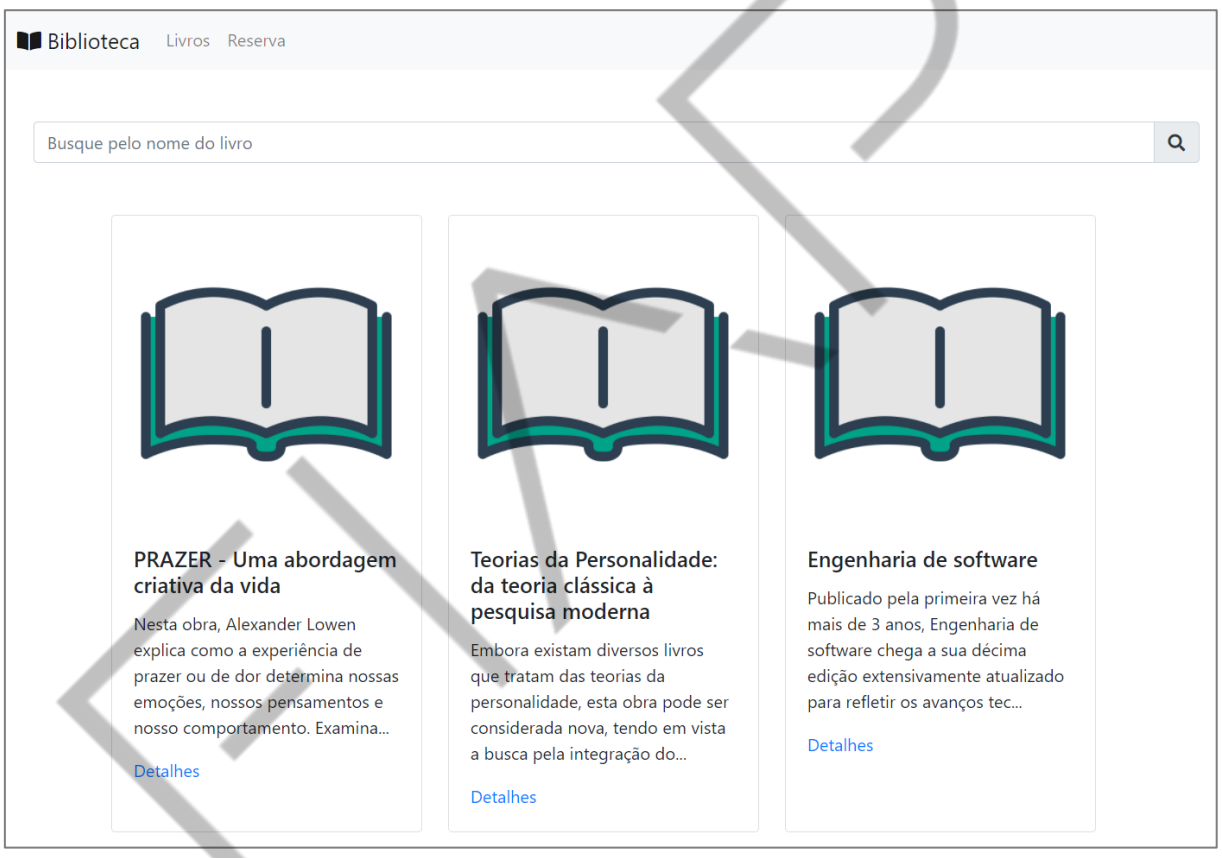

Figura 3.9 – Livros buscados a partir do banco de dados
Fonte: Elaborado pelo autor

Para finalizar essa parte, precisamos apenas criar o acesso ao detalhe do livro, que faremos na sequência.

3.9 Vendo o livro em detalhes

O primeiro passo é trabalharmos para que chegue o identificador do livro através do método. Utilizaremos a url para a passagem de parâmetro. O parâmetro

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

da url deve ser adicionado através do arquivo urls.py. Confira como deixarmos através do código-fonte “Criando os livros com base nos dados”.

```
urlpatterns = [
    path('', views.index, name='index'),
    path('livro/<int:livro_id>', views.livro, name='livro'),
]
```

Código-fonte 3.17 – Criando os livros com base nos dados
Fonte: Elaborado pelo autor

Depois de incluirmos o parâmetro, no qual informamos entre os parênteses angulares o tipo da informação e o nome como nos referenciaremos a ele no método, precisamos adicionar uma maneira de recebê-lo no método do arquivo views.py. Iremos referenciá-lo através de um parâmetro recebido no método ao lado do request. Veja no código-fonte “Recuperando um livro específico do banco de dados”.

```
from django.shortcuts import render, get_object_or_404
from .models import Livro

def index(request):
    dados = {
        'livros': Livro.objects.all()
    }
    return render(request, 'index.html', dados)


def livro(request, livro_id):
    livro = {
        'livro': get_object_or_404(Livro, pk=livro_id)
    }
    return render(request, 'livro.html', livro)
```

Código-fonte 3.18 – Recuperando um livro específico do banco de dados
Fonte: Elaborado pelo autor

Utilizamos o mesmo nome do parâmetro que o determinado na url. Na sequência, importamos do django.shortcuts a função que retornará para nós o objeto de acordo com o id passado ou um erro 404 de objeto não encontrado. Criamos um dicionário e o enviamos com a função render. No código-fonte “Detalhes dinâmicos apresentados no template”, temos o trabalho realizado para exibir os detalhes do livro no template.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<section class="col-12">
    <div class="d-flex flex-row">
        <img src="{% static 'img/book.png'%}" class="img-
fluid img-thumbnail rounded" style="height: 250px"/>
        <div class="flex-column ml-4">
            <h1>{{ livro.titulo }}</h1>
            <h2>{{ livro.autor.nome }}</h2>
        </div>
    </div>
    <div class="d-flex flex-row">
        <p class="col-8">{{ livro.descricao }}</p>
        <ul class="list-group ml-3 col-4">
            <li class="list-group-item">{{ livro.paginas }}
página{{ livro.paginas|pluralize:'s' }}</li>
            <li class="list-group-item">Editora: {{
livro.editora }}</li>
            <li class="list-group-item">Edição: {{
livro.edicao }}</li>
            <li class="list-group-item">Ano da publicação: {{
livro.ano_publicacao }}</li>
            <li class="list-group-item">Idioma: {{
livro.idioma }}</li>
            <li class="list-group-item">ISBN: {{ livro.isbn
}}</li>
        </ul>
    </div>

</section>
```

Código-fonte 3.19 – Detalhes dinâmicos apresentados no template
Fonte: Elaborado pelo autor

O resultado obtido é apresentado através da figura “Detalhes do livro apresentado ao cliente”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

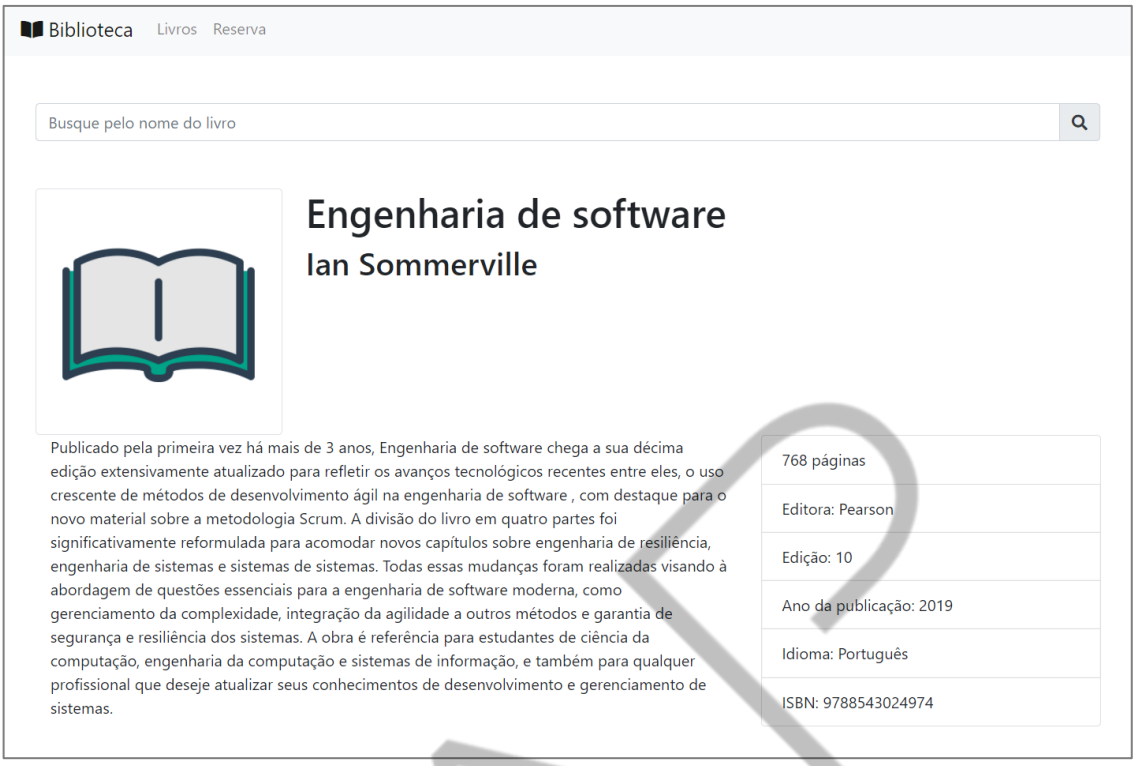

Figura3.10 – Detalhes do livro apresentado ao cliente
Fonte: Elaborado pelo autor

Agora já temos as informações recebidas do banco de dados e apresentadas em nosso sistema para o cliente. Existem muitas coisas a serem feitas, mas trabalharemos aos poucos. Os próximos passos ficam para o capítulo seguinte.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

Django. **Documentação do Django Web Framework**. [s.d.]. Disponível em: https://www.djangoproject.com/. Acesso em: 05 out. 2020.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com